

Deploying a Multi-Tier Application on Kubernetes

A Practical Enterprise Lab Guide for State Management, Declarative Deployment, and Internal Service Mapping

Document Identifier: K8S-DEP-MT-2026

Version: 1.1.0 Hands-On Lab Framework

Target Audience: Engineering Interns, Systems Operators, Infrastructure Trainees

Prerequisites: Module 1-12 Networking Foundations Compliance

Date of Issuance: May 16, 2026

Classification: Internal Engineering Technical Resource

1. Introduction to Multi-Tier Applications on Orchestration Platforms

1.1 Architecture Deconstruction

Modern cloud-native microservices are rarely deployed as monolithic runtime singletons. Instead, they are engineered as decoupled, independent service tiers that separate compute responsibilities into logical layers. A standard multi-tier infrastructure breaks down into three key functional zones:

- **The Client/Presentation Tier:** Typically an HTTP/HTTPS stateless web server container (e.g., NGINX, Apache, or a compiled React/Vue web application wrapper) responsible for delivering assets directly to users or receiving public-facing api connections.
- **The Core Application/Processing Tier:** A stateless API backend processing layer running runtimes like Node.js, Python, Go, or Java. This layer executes transactional logic, evaluates data streams, and serves as an intermediary boundary.
- **The Storage/Data Persistence Tier:** A stateful datastore cluster (e.g., PostgreSQL, MySQL, Redis, MongoDB) that handles long-term transaction state storage, ACID compliance validation, and durable file-system writes.

1.2 The Stateless vs. Stateful Deployment Divide

Orchestrating a multi-tier structure requires you to manage different container lifecycles. Stateless containers (like our web front-end and application API tiers) carry no local data variables. If a web pod crashes, it can be instantly replaced with an identical instance on another host node without loss of data. Scaling up involves spinning up more identical worker pods behind a load balancer.

Stateful containers (like our database tier) present a different set of challenges. A database cannot be treated as a generic, interchangeable processor. It writes long-lived binary records to a underlying block device. If a database pod is destroyed, it must reappear bound to the exact same disk storage volume it was writing to previously, and its network identity must remain predictable so the application tiers do not lose connectivity. This manual will walk you through the mechanisms used to bridge this gap safely.

2. Storage Subsystem Fundamentals: Persistent State Management

2.1 The Lifespan of Ephemeral Container Disks

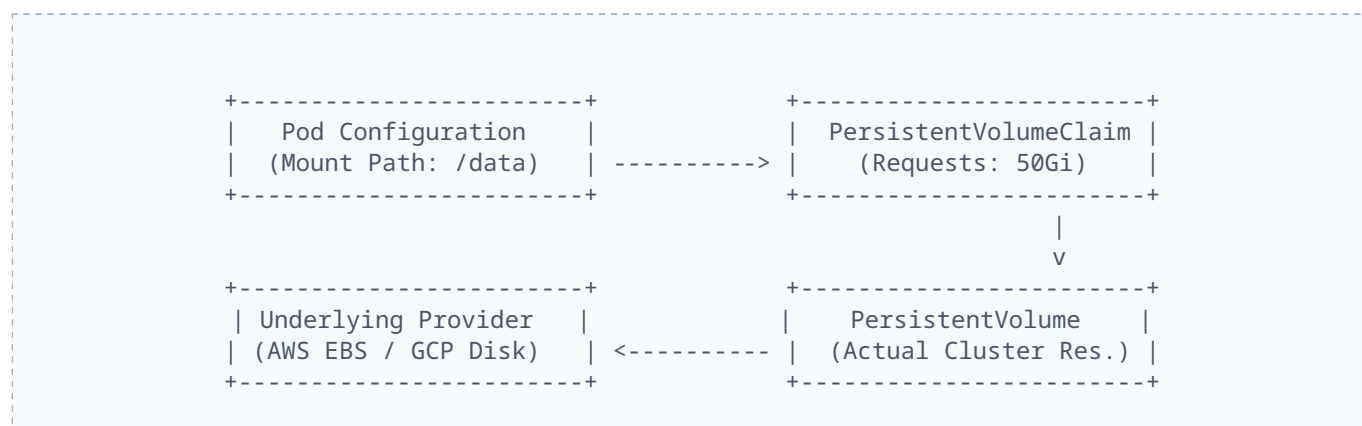
By default, the filesystem inside a standard container is ephemeral. It uses an overlay filesystem structure that shares the host kernel's space. Any data created, modified, or written within a container is stored in a temporary write layer.

If the container process exits, or if the Pod is rescheduled due to a node upgrade or resource constraint, this temporary write layer is permanently wiped out. For databases that store mission-critical logs and transactional tables, relying on this default layer will lead to catastrophic data loss. To prevent this, Kubernetes uses an abstracted storage management system.

2.2 Volumes, PersistentVolumes (PV), and PersistentVolumeClaims (PVC)

To safely decouple physical infrastructure from application configurations, Kubernetes splits its storage model into three distinct components:

- **Volumes:** A directory accessible to containers within a Pod. The life of a standard volume is tied directly to the lifecycle of the enclosing Pod. While useful for sharing temporary data between containers in the same Pod, it cannot protect data across Pod recreations.
- **PersistentVolume (PV):** A cluster-wide storage resource provisioned by an administrator or dynamically generated via a Cloud Storage Class. It is an independent cluster object that maps directly to real physical storage, such as an AWS EBS block device, a Google Cloud Persistent Disk, an NFS share, or a local NVMe disk array. Its lifecycle is entirely independent of any Pod using it.
- **PersistentVolumeClaim (PVC):** A developer's request for storage. A PVC acts as an abstract ticket that allows an application to request specific storage parameters (such as size, like 50Gi, and access modes, like ReadWriteOnce) without needing to know anything about the underlying cloud provider or physical storage hardware.



2.3 Understanding Storage Access Modes

When creating a storage request, developers must define how the volume should be attached to the underlying hosts. Kubernetes defines three primary access modes:

Access Mode String	Abbreviation	Operational Mechanism
<code>ReadWriteOnce</code>	RWO	The volume can be mounted with read-write permissions by a single Kubernetes worker node at any given time. Ideal for traditional block storage like AWS EBS.
<code>ReadOnlyMany</code>	ROM	The volume can be mounted with read-only permissions simultaneously by many different worker nodes across the cluster. Great for distributing static assets or configuration files.
<code>ReadWriteMany</code>	RWM	The volume can be mounted with read-write permissions simultaneously by many different worker nodes. Requires shared network file systems like NFS, AWS EFS, or GlusterFS.

3. Blueprinting the Storage Layer: PV and PVC Manifests

3.1 The Concrete PersistentVolume Manifest

Below is a production-grade declarative manifest for a `PersistentVolume` using a local host mount directory. This manifest registers a durable block of 20 Gigabytes of storage with the cluster control plane.

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: database-storage-pv
  labels:
    tier: storage-backend
spec:
  capacity:
    storage: 20Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: manual
  hostPath:
    path: "/mnt/enterprise-data/postgres"
```

3.2 The Developer PersistentVolumeClaim Manifest

To request and consume the storage resource provisioned above, you must create a matching `PersistentVolumeClaim`. The cluster control plane automatically evaluates active PVC requests and binds them to available PVs that match the requested size, storage class, and access mode criteria.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: database-storage-pvc
  namespace: production
spec:
  accessModes:
    - ReadWriteOnce
  volumeMode: Filesystem
  resources:
    requests:
      storage: 20Gi
  storageClassName: manual
```

Important Reclaim Policy Note: In the PV manifest above, setting `persistentVolumeReclaimPolicy: Retain` protects your data from accidental deletion. When a developer deletes a PVC, the corresponding PV enters a "Released" state rather than being instantly wiped. This allows a cluster administrator to manually recover and salvage data from the underlying disk before freeing up the resource.

4. Blueprinting the Database Tier Deployment

4.1 Architecture of the Stateful Target

The database tier serves as the data foundation for our multi-tier application. In this lab, we will use a production-hardened PostgreSQL container instance. To configure this tier correctly, we need to apply environmental access credentials and mount our newly provisioned `PersistentVolumeClaim` directly into the container's active write directory.

4.2 Comprehensive Database Deployment Manifest

Save the following declarative blueprint as `database-deployment.yaml`. Note how it defines resource parameters, maps environment variables for access control, and attaches the storage volume to the correct application paths.

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: database-tier-deployment
  namespace: production
  labels:
    tier: database
spec:
  replicas: 1
  selector:
    matchLabels:
      app: relational-db
  template:
    metadata:
      labels:
        app: relational-db
        tier: database
    spec:
      containers:
        - name: postgresql-container
          image: postgres:15.3-alpine
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 5432
              name: dbport
          env:
            - name: POSTGRES_DB
              value: enterprisedb
            - name: POSTGRES_USER
              value: cloud_admin
            - name: POSTGRES_PASSWORD
              value: VaultSecuredPass2026
          resources:
            requests:
              memory: "256Mi"
              cpu: "250m"
            limits:
              memory: "512Mi"
              cpu: "500m"
          volumeMounts:
            - name: active-postgres-volume
              mountPath: /var/lib/postgresql/data
      volumes:
        - name: active-postgres-volume
          persistentVolumeClaim:
            claimName: database-storage-pvc

```

5. Connecting Tiers: Database Internal Service Networking

5.1 Isolating the Storage Layer

The database tier must never be exposed directly to the outside public internet or external cluster gateways. To minimize your attack surface, the database should only be reachable by other workloads running inside the cluster.

To implement this isolation, we will configure a dedicated **ClusterIP Service** for our database layer. This assigns a stable, internal-only IP address and a corresponding local DNS entry that our API tier can use to safely route its transactions.

5.2 The Database Service Manifest

Save the following manifest configuration as `database-service.yaml`. This configures a stable network endpoint that maps directly to our database containers over port 5432.

```
apiVersion: v1
kind: Service
metadata:
  name: database-link-service
  namespace: production
  labels:
    tier: database
spec:
  type: ClusterIP
  ports:
    - port: 5432
      targetPort: dbport
      protocol: TCP
  selector:
    app: relational-db
```

5.3 Internal Service Resolution Path

Once this manifest is applied to the cluster, the internal CoreDNS engine instantly registers a matching search pointer. Any stateless application tier running anywhere inside the `production` namespace can now securely connect to the database simply by pointing its configuration strings to the following short hostname:

```
database-link-service:5432
```

6. Blueprinting the Application API Tier Deployment

6.1 Designing the Stateless Processing Engine

With our database tier safely configured and isolated, we can now deploy the application processing tier. This tier runs our core business logic and needs to communicate directly with the backend database.

Because this tier is stateless, we can safely scale it out to multiple replicas. This allows us to handle increased user traffic and provides automatic redundancy if an individual application container or worker node experiences a failure.

6.2 Comprehensive Application Deployment Manifest

Save the following declarative manifest configuration as `app-tier-deployment.yaml`. Note how we inject the database's internal DNS hostname directly into the application's environment variables, allowing the API code to locate the database connection string automatically.

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: app-tier-deployment
  namespace: production
  labels:
    tier: application
spec:
  replicas: 3
  selector:
    matchLabels:
      app: business-api
  template:
    metadata:
      labels:
        app: business-api
        tier: application
    spec:
      containers:
        - name: api-engine-container
          image: node:18-alpine
          command: ["sh", "-c"]
          args:
            - |
              cat << 'EOF' > server.js
              const http = require('http');
              const server = http.createServer((req, res) => {
                res.writeHead(200, {'Content-Type': 'application/json'});
                res.end(JSON.stringify({
                  status: "Online",
                  tier: "Application API Processing Layer",
                  databaseTarget: process.env.DB_HOST
                }));
              });
              server.listen(8080);
              EOF
              node server.js
          ports:
            - containerPort: 8080
              name: apiport
          env:
            - name: DB_HOST
              value: "database-link-service"
            - name: DB_USER
              value: "cloud_admin"
            - name: DB_PASS
              value: "VaultSecuredPass2026"
      resources:
        requests:
          memory: "128Mi"
          cpu: "100m"
        limits:
          memory: "256Mi"
          cpu: "250m"

```

7. Scaling and Inter-Tier Communication

7.1 Dynamic Load Balancing Mechanics

Because we configured the application tier to run 3 concurrent replicas, the cluster control plane distributes these Pods across the available worker nodes. To efficiently balance traffic across these replicas, we must deploy an upstream Service.

This service manages an internal virtual ClusterIP address that acts as a single gateway for the API layer. When traffic hits this address, the host's kernel load balances the requests across the underlying application replicas using hashed routing rules, ensuring optimal resource usage and preventing any single container from becoming a bottleneck.

7.2 The Application Service Manifest

Save the following declarative blueprint as `app-tier-service.yaml` to establish the load balancing layer for the API tier.

```
apiVersion: v1
kind: Service
metadata:
  name: app-tier-link-service
  namespace: production
  labels:
    tier: application
spec:
  type: ClusterIP
  ports:
    - port: 8080
      targetPort: apiport
      protocol: TCP
  selector:
    app: business-api
```

8. Edge Routing: Public Access via NodePort or Ingress

8.1 Exposing the Entry Point

At this stage, our database tier and application API tier are fully initialized, linked, and isolated within our internal cluster network. The final step is to build an external entry point that allows outside clients and end-users to securely connect to our application.

To achieve this, we can use a **NodePort Service** or configure a Layer 7 **Ingress Controller**. For this lab manual, we will deploy a standardized NodePort configuration that maps our application's internal entry point directly to a dedicated port across all physical worker nodes, making it accessible to external networks.

8.2 The Public Presentation Layer Manifest

Save the following configuration blueprint as `public-edge-service.yaml`. This exposes our application tier to external users on port `32080` across every node interface in the cluster.

```
apiVersion: v1
kind: Service
metadata:
  name: public-edge-gateway
  namespace: production
  labels:
    tier: edge
spec:
  type: NodePort
  ports:
    - port: 80
      targetPort: apiport
      nodePort: 32080
      protocol: TCP
  selector:
    app: business-api
```

9. Step-by-Step Deployment Execution Plan

9.1 Standard Cluster Application Ordering

When deploying a multi-tier application, you must follow a precise sequence. Deploying an application tier before its database dependencies or storage configurations are ready can cause connection errors and push your containers into a failing state. Follow this step-by-step execution order:

1. Step 1: Create the target environment namespace

```
kubectl create namespace production
```

2. Step 2: Initialize the physical storage configurations

```
kubectl apply -f pv-volume-blueprint.yaml  
kubectl apply -f pvc-claim-blueprint.yaml
```

3. Step 3: Deploy the isolated database tier and verify its storage binding

```
kubectl apply -f database-deployment.yaml  
kubectl apply -f database-service.yaml
```

4. Step 4: Deploy the application API processing tier

```
kubectl apply -f app-tier-deployment.yaml  
kubectl apply -f app-tier-service.yaml
```

5. Step 5: Bind the external public access edge port

```
kubectl apply -f public-edge-service.yaml
```

10. Lab Verification and Live Environment Diagnostics

10.1 Auditing Storage Volume Bindings

Before launching a verification request, confirm that your storage claims have successfully bound to your storage volumes. Run the following audit command:

```
kubectl get pvc -n production
```

Verify that the status column displays **Bound**. If the claim stays trapped in a **Pending** state, run the describe command to look for underlying storage allocation errors:

```
kubectl describe pvc database-storage-pvc -n production
```

10.2 Verifying Multi-Tier End-to-End Connectivity

Once all deployments are up and running, find the public IP address of one of your cluster worker nodes and execute a live validation test using **curl** against your configured edge port:

```
curl -v http://<WORKER-NODE-PUBLIC-IP>:32080
```

A successful deployment will return a valid JSON data payload confirming that the presentation layer is online and the application tier can successfully resolve your internal database link service.